

Programming for Essential Digital Skills, Chapter 8

Christoph Walsh

2026

Tabular Data

- Data are often in tabular format.
- Columns represent variables and rows indicate observations, or records.
- Example data on firm sales from two regions for different dates:

date	region	sales
2024-06-01	north	342
2024-06-01	south	251
2024-06-02	north	489
2024-06-02	south	372
2024-06-03	north	589
2024-06-03	south	492

Comma-Separated Value Files

- Tabular data are often be stored in comma-separated value (CSV) format.
- These have the file extension `.csv`.
- The contents of a file containing the data on the previous slide would look like:

```
date,region,sales
2024-06-01,north,342
2024-06-01,south,251
2024-06-02,north,489
2024-06-02,south,372
2024-06-03,north,589
2024-06-03,south,492
```

Commas in the Data

- Sometimes data points contain commas.
- For example, in strings (north,west) or numbers with comma decimal separators (1,23).
- To prevent these commas being interpreted as column separators, we add double quotes:

```
"date", "regions", "revenue"  
"2025-07-01", "north,west", "3423,40"  
"2025-07-01", "south,east", "4352,45"  
"2025-07-02", "north,west", "3635,30"  
"2025-07-02", "south,east", "4252,53"  
"2025-07-03", "north,west", "3734,35"  
"2025-07-03", "south,east", "4742,67"
```

Reading CSV Files into Python

- To read CSV files into Python, we use the pandas module.
- We will learn how to read in data using the file `HousingPrices-Amsterdam-August-2021.csv` (from Canvas).
- Importantly, the `.csv` file must be in the same folder on your computer as your Python script.
- Following convention, we load the pandas module as `pd`.
- It is also conventional to read in your data as `df` (for DataFrame).

```
import pandas as pd
df = pd.read_csv("HousingPrices-Amsterdam-August-2021.csv")
```

Data Cleaning

- Often when we read in data it's not ready to be analyzed immediately.
- We need to perform some cleaning steps to get it ready.
- For example, we might need to:
 - ▶ Drop some variables or rows.
 - ▶ Create new variables from existing ones.
 - ▶ Change variable names.

Dropping Variables

- The first variable, Unnamed: 0 is just a row index from 1 to 924.
- This is useless so we can drop it from the data using the drop method:

```
df.drop('Unnamed: 0', axis=1, inplace=True)
```

- Here, axis=1 means I want to drop a column (it would be 0 for a row)
- inplace=True means it modifies df in place.

Dropping Rows with Missing Vales

- To see how many rows have missing values for each variable:

```
df.isna().sum()
```

```
Address      0  
Zip          0  
Price        4  
Area         0  
Room         0  
Lon          0  
Lat          0  
dtype: int64
```

Dropping Rows with Missing Vales

- To see the rows with missing values for price:

```
df[df['Price'].isna()]
```

	Address	Zip	Price	...	Room	L
73	Falckstraat 47 A, Amsterdam	1017 VV	NaN	...	3	4.8974
321	Haarlemmerweg 705, Amsterdam	1067 HP	NaN	...	12	4.7878
610	Zeeburgerkade 760, Amsterdam	1019 HT	NaN	...	3	4.9450
727	Suikerplein 16, Amsterdam	1013 CK	NaN	...	3	4.8809

```
[4 rows x 7 columns]
```

Dropping Rows with Missing Vales

- To drop rows with a missing value for price:

```
df = df[~df['Price'].isna()]
```

- Here the tilde ~ means not, i.e. the rows where price is not missing.
- To drop all rows containing missing values from any variable:

```
df.dropna(inplace=True)
```

Creating New Variables

- To create a new variable from existing variables:

```
df['price_per_sqm'] = df['Price'] / df['Area']
```

Renaming Variables

- Sometimes we want to change variable names.
- We can see all the variable names using the command `df.columns`:

```
df.columns
```

```
Index(['Address', 'Zip', 'Price', 'Area', 'Room', 'Lon', 'Lat',  
      'price_per_sqm'],  
      dtype='str')
```

- We rename using a dictionary (keys are the old names, values are the new names):

```
df.rename(columns={"Zip" : "postcode", "Room" : "num_rooms"},  
         inplace=True)
```

Renaming Variables

- To convert all variable names to lower case:

```
df.columns = df.columns.str.lower()
```

Describing Data

- To get basic summary statistics of all variables:

```
df.describe()
```

	price	area	...	lat	price_per_sqm
count	9.200000e+02	920.000000	...	920.000000	920.000000
mean	6.220654e+05	95.607609	...	52.363271	6479.852299
std	5.389942e+05	56.849699	...	0.024054	2219.265218
min	1.750000e+05	21.000000	...	52.291519	2430.555556
25%	3.500000e+05	60.000000	...	52.351925	4649.086379
50%	4.670000e+05	83.000000	...	52.364499	6578.947368
75%	7.000000e+05	113.000000	...	52.377545	7768.912530
max	5.950000e+06	623.000000	...	52.423805	25252.808989

```
[8 rows x 6 columns]
```

Summary Statistics

- To get the mean of a specific variable:

```
df['price'].mean()
```

```
np.float64(622065.4195652173)
```

Distribution of Values (Most Common to Least Common)

```
df['num_rooms'].value_counts()
```

num_rooms

3 330

4 201

2 191

5 97

6 42

7 19

1 17

8 11

9 6

13 2

10 2

14 2

Name: count, dtype: int64

Distribution of Values (Sorted by Value)

```
df['num_rooms'].value_counts().sort_index()
```

num_rooms

1 17

2 191

3 330

4 201

5 97

6 42

7 19

8 11

9 6

10 2

13 2

14 2

Name: count, dtype: int64

Distribution of Values (in Proportions)

```
df['num_rooms'].value_counts(normalize=True)
```

```
num_rooms
3      0.358696
4      0.218478
2      0.207609
5      0.105435
6      0.045652
7      0.020652
1      0.018478
8      0.011957
9      0.006522
13     0.002174
10     0.002174
14     0.002174
Name: proportion, dtype: float64
```

Subsetting

- Suppose I want the average price of houses with 4 rooms.
- I first get the subset of the data with 4 rooms.
- I then get the mean of price from that.

```
df[df['num_rooms'] == 4]['price'].mean()
```

```
np.float64(609845.0945273632)
```

Subsetting

- Suppose I want the average price of houses with 4 rooms and have at least 100 square meters:

```
df[(df['num_rooms'] == 4) & (df['area'] >= 100)][['price']].mean()  
  
np.float64(859761.7764705883)
```

Subsetting

- Suppose I want the average price of houses with 4 rooms or have at least 100 square meters:

```
df[(df['num_rooms'] >= 4) | (df['area'] >= 100)][['price']].mean()  
  
np.float64(847846.6853932585)
```

Group by Operations

- Suppose I want the average price for houses with:
 - ▶ 1 room
 - ▶ 2 rooms
 - ▶ 3 rooms
 - ▶ ...

```
df.groupby('num_rooms')['price'].mean().round()
```

```
num_rooms
1      394529.0
2      383479.0
3      512416.0
4      609845.0
5      845076.0
6      919929.0
7     1394737.0
8     1751636.0
9     1450000.0
```

Applying Functions

- Suppose we want to create a new variable which is the 4-digit postcode.
- For a single postcode, we could apply the following function to get it:

```
def get_four_digits(x):  
    return x[:4]
```

```
df['postcode'][0]
```

```
'1091 CR'
```

```
get_four_digits(df['postcode'][0])
```

```
'1091'
```

Applying Functions

- To apply this function to all postcodes to create a new variable:

```
df['four_digit_postcode'] = df['postcode'].apply(get_four_digits)
```

Lambda Functions

- For short and simple functions, we can create a temporary “lambda” function instead:

```
df['four_digit_postcode'] = df['postcode'].apply(lambda x: x[:4])
```

Exporting Data

- After cleaning the data we may want to export it out of Python.
- We can do that with:

```
df.to_csv("HousingPrices-Amsterdam-August-2021-cleaned.csv",  
         index=False)
```

- Avoid writing over the raw data. Better to save to a new file.
- The option `index=False` avoids creating an extra column with row indices.